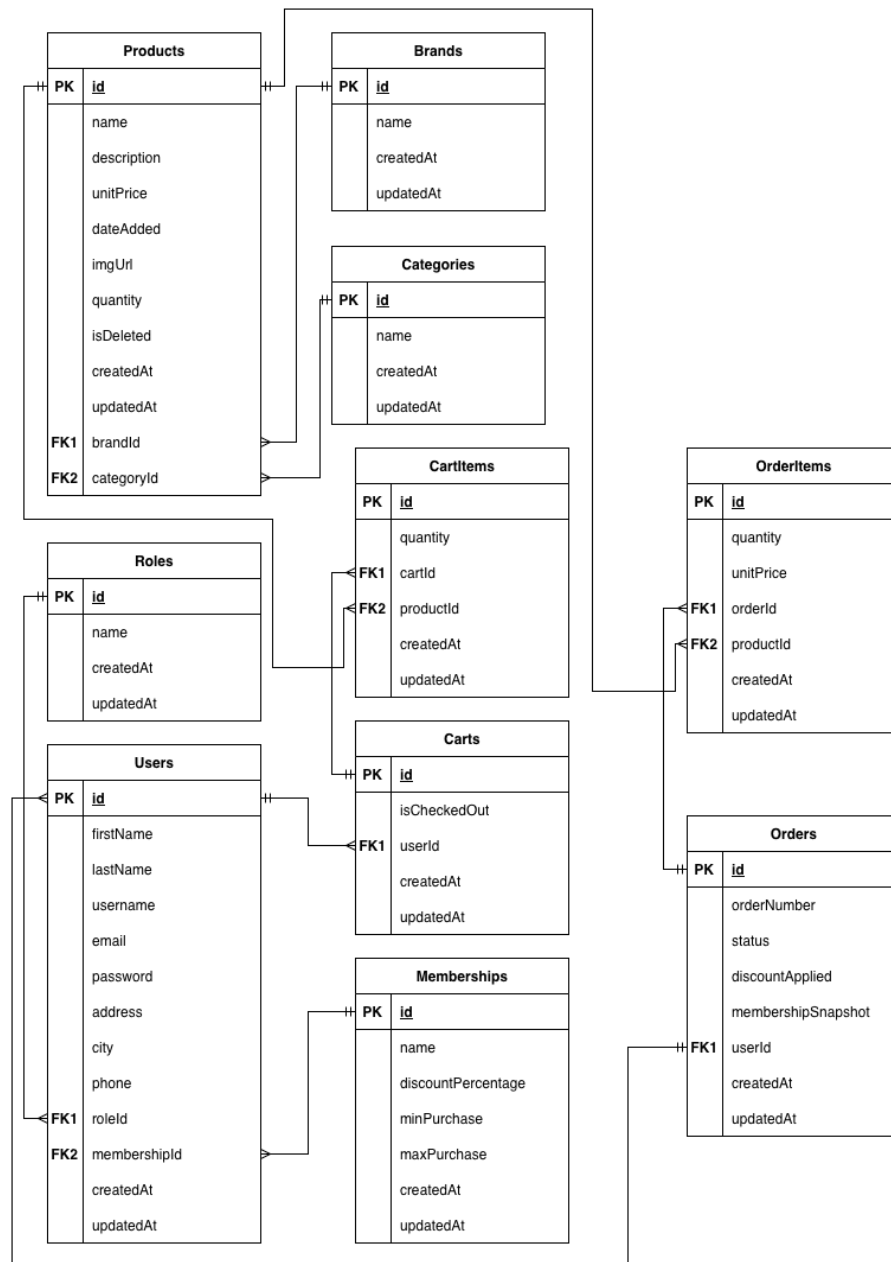


# Reflection Report

## 1. Database ERD

The diagram below has been made using draw.io. It explains the complete database schema which is designed in 3NF. This means that the database is structured so that all data is stored in exactly one place, no duplication and no column that depend on anything other than the PK.



## 2. Table Relationships

### Roles → Users

One-to-many. A Role can be assigned to many Users, but each User has exactly one Role (stored as roleId FK on Users). This separates access control from user data.

### Memberships → Users

One-to-many. A Membership tier (Bronze, Silver, Gold) can apply to many Users, but each User holds exactly one active membership at a time (membershipId FK on Users). The tier is recalculated automatically after every checkout.

### Brands → Products

One-to-many. A Brand can have many Products, but each Product belongs to exactly one Brand (brandId FK on Products). Separating brands into their own table eliminates redundancy.

### Categories → Products

One-to-many. A Category can contain many Products, but each Product belongs to exactly one Category (categoryId FK on Products).

### Users → Carts

One-to-many. A User can have multiple Carts over time, but each Cart belongs to one User (userId FK on Carts). Only the cart where isCheckedOut = false is the active cart.

### Carts → CartItems

One-to-many. A Cart can contain many CartItems, but each CartItem belongs to exactly one Cart (cartId FK on CartItems).

### Products → CartItems

One-to-many. A Product can appear in many CartItems across different carts, but each CartItem references exactly one Product (productId FK on CartItems).

Users  $\rightarrow$  Orders

One-to-many. A User can have many Orders over time, but each Order belongs to one User (userId FK on Orders). Orders are created when a Cart is checked out.

## Orders → OrderItems

One-to-many. An Order can contain many OrderItems, but each OrderItem belongs to exactly one Order (orderId FK on OrderItems). The unit price is stored on each OrderItem to preserve the price at the time of purchase.

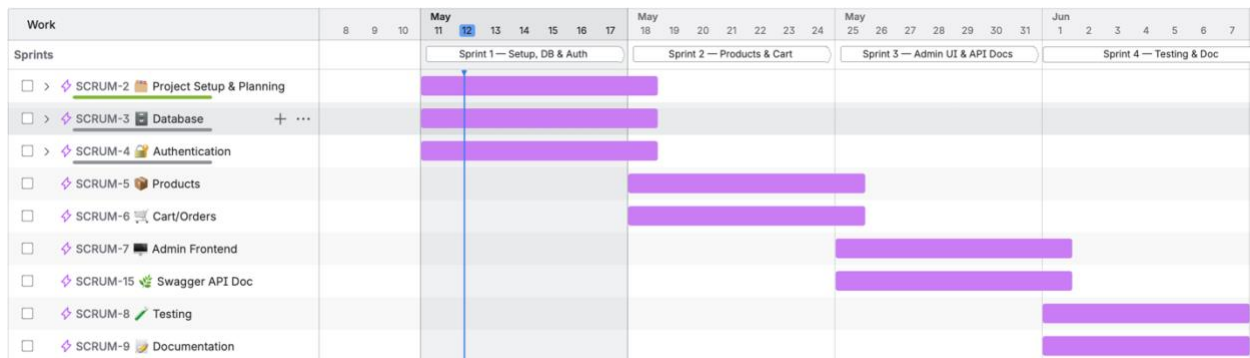
Products  $\rightarrow$  OrderItems

One-to-many. A Product can appear in many OrderItems across different orders, but each OrderItem references one Product (productId FK on OrderItems).

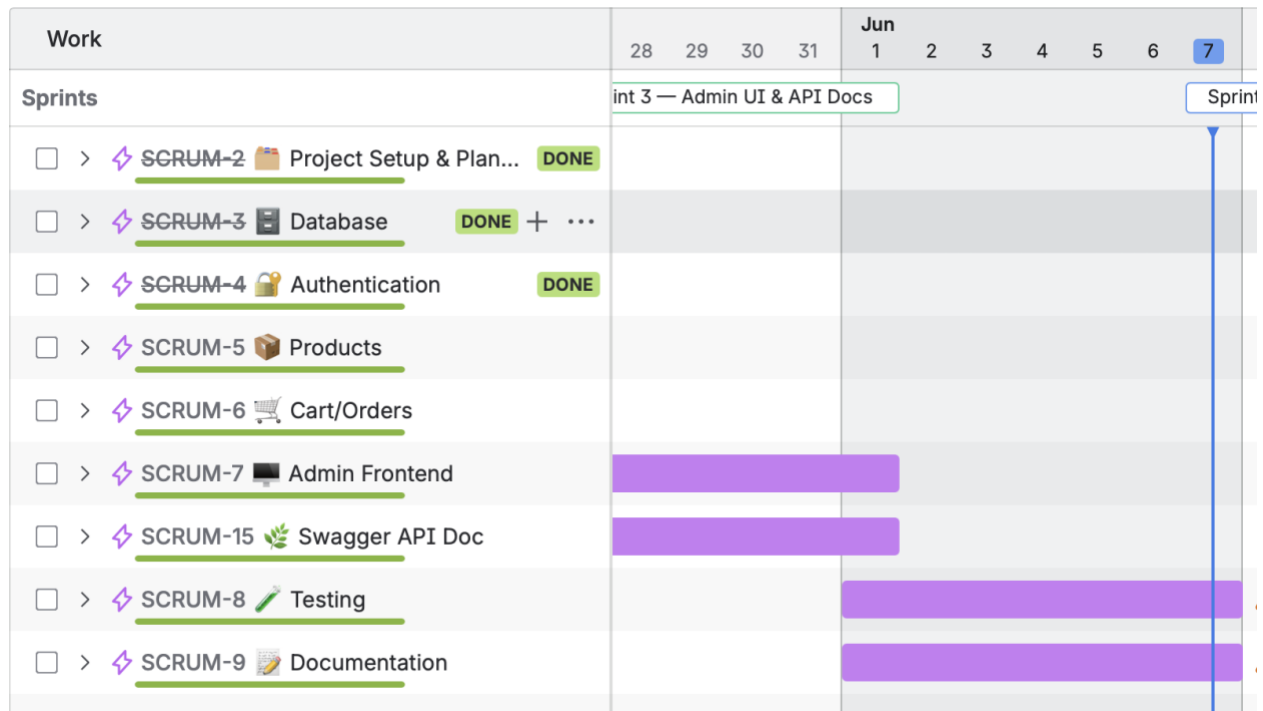
### 3. Jira Roadmap

The project was worked on over four sprints using Jira. One sprint lasted one week and in total I had 9 epics. It was divided into in 3 epics for Sprint 1, then 2 epics for the remaining sprints. The timeline below showed the planned timeline from the start of the project to the completed state at the after finishing Sprint 4.

## Sprint Planning:



## Timeline after finishing Sprint 4:



## 4. Project Reflection

### 4.1 Project Progression

The project started on May 11th with Jira setup and initial planning. Getting the board organized from day one made a real difference. Having epics like Database, Auth, Products, Cart, Admin Frontend, Testing and Documentation added to specific sprints gave the project a clear structure so I know what to work on each day.

The first full day of work went into designing the database ERD using draw.io. This turned out to be one of the most important steps in the whole project. Getting the third normal form right from the beginning meant there were no structural problems to fix later. A lot of time was spent thinking what should be a FK for what, what is redundant and in general how to just keep the database ERD structured with 3NF.

Backend development started on May 13th with Sequelize setup and MySQL connection. The decision to keep one model file per table and combine all associations in models/index.js paid off as it kept things clean and easy to navigate throughout the project. Service files were added next and it was made to follow DRY (Don't Repeat Yourself) principles, which meant controllers stayed focused on request/response handling without any duplicated database logic.

The cart and checkout endpoint on May 19th was a milestone. The checkout creates a unique 8-character order number, captures the membership tier at the time of purchase, stores the unit price on each order item so future price changes don't affect historical orders, and recalculates the user's membership tier afterwards. All of this was tested in Postman and verified in MySQL Workbench.

Products, raw SQL queries, orders and search were worked on between May 24th and 26th. The raw SQL requirement for product listings and search added complexity but was manageable once the query structure was clear. Role-based filtering where admins see soft-deleted products and normal users do not, which was implemented cleanly at the middleware level.

The admin frontend was the largest single piece of work. Starting May 28th, a full Express/EJS/Bootstrap application was built on port 3001 with session-based authentication. All data flows exclusively through the backend API, no service files or direct database access are used in the frontend. Views were built for Products, Brands, Categories, Roles, Users and Orders. All had consistent table layouts.

A bit late, but Swagger documentation which was a part of Sprint 3, was not configured before June 5<sup>th</sup>, as it took a lot more time finishing admin frontend. It is covering all endpoints with full request and response schemas. Which overlapped late with Sprint 4, and then Sprint 4 was started and completed from June 6<sup>th</sup>-7<sup>th</sup> which included testing and documentation. Another reason for a big break in Sprint 3 and late completion of Sprint 4, was due to personal reasons, which made me unable to work on the project.

## 4.2 Challenges Faced

The most time-consuming challenge was the admin frontend. EJS templating and Bootstrap were technologies I had not practiced a lot before, and the learning curve was steeper than expected.

Sprint overlap was another challenge. The Authentication epic spilled from Sprint 1 into Sprint 2, and the admin frontend ran through both Sprint 3 and Sprint 4. In hindsight, the admin frontend should have had a larger time estimate.

## 5. Example .env File

Below you can see an example of the environment configuration file used in the backend and frontend. Real credentials are not included.

### Backend env\_example file

```
DB_HOST=localhost
DB_USER=root
DB_PASSWORD=
DB_NAME=
DB_PORT=3306
JWT_SECRET=
PORT=3000
```

### Frontend env\_example file

```
PORT=3001
BACKEND_URL=http://localhost:3000
SESSION_SECRET=
```

## 6. References

**Bootstrap documentation:** <https://getbootstrap.com>

**Background image:** <https://unsplash.com/photos/a-building-with-glass-windows-3VThnphFT4o>

**Swagger:** <https://swagger.io/docs/>

**Sequelize documentation:** <https://sequelize.org/docs/v6/>

**Express documentation:** <https://expressjs.com>

**JWT documentation:** <https://jwt.io>

**Claude AI:** Used as a guided assistant throughout development. This included getting a logical approach to structuring and planning the project in Jira, such as understanding the right order to approach things like starting with planning and database design before moving on to building endpoints, then finishing with testing and documentation. Claude was also used during development to help understand errors, for example by asking questions like "explain what is causing this error" after I have written some code with syntax errors. This was to get a better understanding of what was going wrong so I could try to find a solution for my error. If I needed suggestions, it was carefully reviewed and tested before use, and no code or approach was adopted without fully understanding its purpose and functionality.